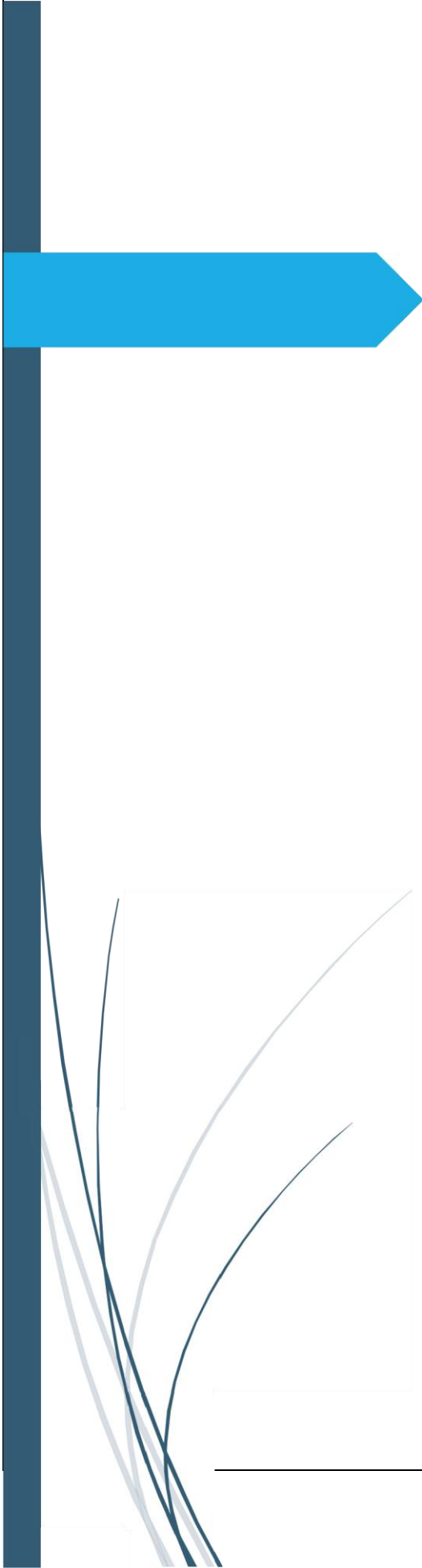




FALL SEMESTER 2025-26

Analog Circuits [BECE206L]
**ANALOG CIRCUITS COURSE
PROJECT REPORT**

PROJECT TITLE:

SMART NOISE CANCELLATION SYSTEM

Group Members:

Vatsa Jamar

Avik Raj

Abhinav Gupta

Aditya

TABLE OF CONTENTS

Sl. No.	Topics	Page No.
	List of Figures	3
1.	INTRODUCTION	5
1.1	Literature Review	6
1.2	Aim	8
1.3	Objectives	8
2.	HARDWARE COMPONENTS USED	9
3.	WORK DONE	8
3.1	Implementation 1: Analog Active Noise Cancellation ("Hardware Version")	9
	3.1.1 System Block Diagram	10
	3.1.2 Circuit Design and Simulation	11
	3.1.3 Physical Prototype (Analog)	12
3.2	Implementation 2: Digital IoT Noise Monitoring ("Software Version")	14
	3.2.1 System Architecture	14
	3.2.2 Physical Prototype (Digital)	15
4.	SOFTWARE CODING	15

4.1	ESP32 Firmware (Data Acquisition & Encryption)	15
4.2	MATLAB Script (Cloud Data Analysis & Visualization)	20
5.	RESULTS & DISCUSSION	23
5.1	Results: Analog ANC Simulation	23
5.2	Results: Digital IoT Monitoring Dashboard	24
5.3	Discussion: A Comparative Analysis	25
6.	FUTURE WORK	26
7.	REFERENCES	27

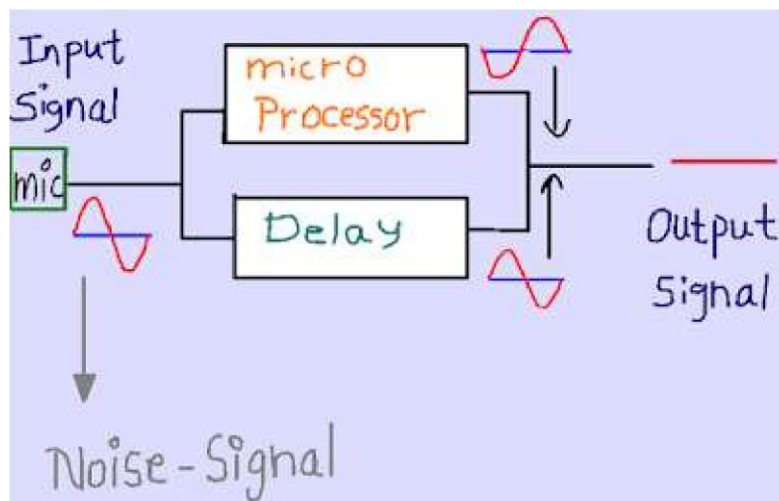
LIST OF FIGURES

Figure No.	Figure Title	Page No.
Fig 1.1	Principle of Active Noise Cancellation (Destructive Interference)	5
Fig 3.1	Block Diagram of the Analog Feedforward ANC System	10
Fig 3.2	Analog ANC Circuit Schematic (Multisim Design)	11
Fig 3.3	Physical Prototype (Top View - Digital/Analog)	12

	Components)	
Fig 3.4	Physical Prototype (Bottom View - Soldering)	13
Fig 5.1	Simulation Results from Four-Channel Oscilloscope (XSC2)	23
Fig 5.2	ThingSpeak Dashboard with Processed Noise Data	25

1. INTRODUCTION

Acoustic noise is one of the most pervasive and challenging forms of environmental pollution. In industrial, urban, and even domestic settings, unwanted sound from machinery, transportation, and appliances can cause hearing damage, reduce productivity, and significantly decrease quality of life. Traditionally, noise control has been dominated by passive methods—using heavy, bulky materials like foam and insulation to absorb or block sound. While effective for high frequencies, these methods are largely impractical for low-frequency noise.



(Fig 1.1)

This challenge gives rise to **Active Noise Control (ANC)**, a modern solution that uses electronics to fight noise with noise. The core principle, shown in Fig 1.1, is **destructive interference**. An ANC system uses a microphone to "listen" to the unwanted noise, an electronic circuit to instantly process it, and a speaker to produce an identical sound wave that is 180 degrees out of phase (an "anti-noise"). When the original noise and the anti-noise meet, they cancel each other out, creating a "zone of quiet."

Fig 1.1: Principle of Active Noise Cancellation (**Destructive Interference**)

This project combines concepts of Analog Circuits with IOT , Cryptography and Signal Processing. It is a sophisticated **digital IoT noise cancellation and monitoring system**. This system uses an ESP32 microcontroller to *measure* noise, *securely end to end encrypt* the data using AES and additional encryption , process the data and transmit it to the Thingspeak cloud based dashboard . A MATLAB script then performs data analysis, decryption, and visualization. This approach prioritizes intelligence, data logging, and security.

1.1 Literature Review

A review of existing literature was conducted to understand the foundational principles of both active noise cancellation and IoT-based environmental monitoring.

1. Analog Active Noise Control:

The concept of analog ANC is well-established. Early systems, as detailed by S. M. Kuo and D. R. Morgan in Active Noise Control Systems, often relied on feedforward analog circuits. These circuits, typically built with operational amplifiers, are designed to create a specific, fixed phase inversion and gain. As explored by researchers, the primary advantage is the extremely low latency (processing delay), which is critical for cancelling high-frequency sounds where phase accuracy is paramount. However, their main limitation is their fixed-filter nature. They are tuned for a specific noise profile and cannot adapt if the noise changes in frequency or character.

2. Digital Active Noise Control (FxLMS):

The limitations of analog ANC led to the dominance of digital systems, most notably those using the Filtered-x Least Mean Squares (FxLMS) algorithm. This algorithm, a cornerstone of adaptive signal processing, allows a digital processor (like a DSP or microcontroller) to continuously "listen" to the residual (uncancelled) noise and "learn" or adapt its anti-noise signal in real-time to be more effective. The work of B. Widrow and S. D. Stearns on Adaptive Signal Processing provides the theoretical foundation for these systems.

3. IoT-based Signal Monitoring (ThingSpeak):

Parallel to noise cancellation, the field of noise monitoring has been revolutionized by the Internet of Things. Platforms like ThingSpeak allow for the easy collection, visualization, and analysis of sensor data from any-where in the world. As documented by numerous case studies, low-cost microcontrollers like the ESP32 can be paired with sensors to create distributed monitoring networks. This allows for large-scale data collection on noise pollution, which is essential for urban planning and industrial regulation.

4. Secure IoT Data Transmission (AES):

A significant challenge in IoT systems is data security. Sending sensor data "in the clear" over Wi-Fi is a major security risk. The Advanced Encryption Standard (AES) has become the industry-standard solution. AES is a symmetric-key algorithm that is both highly secure and computationally efficient enough to run on modern microcontrollers like the ESP32. Implementing AES-CBC (Cipher Block Chaining) ensures that data sent to the cloud is protected from eavesdropping and tampering, which is a critical requirement for any professional or scalable IoT application.

This project sits at the intersection of these fields, directly comparing a classic analog ANC design

(Literature 1) with a modern, secure IoT monitoring system (Literatures 3 & 4).

1.2 Aim

The aim of this project is to develop a **secure IoT sound sensor data acquisition , analog based ANC as well as its monitoring system** using ESP32 and ThingSpeak cloud platform, with advanced data protection and real-time analytics capabilities via a cloud platform.

1.3 Objectives

To achieve the stated aim, the following objectives were established:

1. To design and simulate a feedforward analog ANC circuit using a BJT pre-amplifier, an op-amp phase inverter, and an op-amp summing amplifier, using NI Multisim.
2. To analyze the simulation results to confirm the theoretical validity of the analog circuit's noise-canceling properties.
3. To construct a physical prototype of the analog ANC circuit on perfboard.
4. To develop a digital monitoring system using an ESP-WROOM-32 microcontroller, a sound sensor , and a stable power supply.
5. To write and deploy ESP32 firmware that captures sound data, securely encrypts it using the 128-bit AES algorithm, and transmits the encrypted data to a ThingSpeak cloud channel via Wi-Fi.
6. To implement a geo-location feature in the firmware to update the ThingSpeak channel's location.
7. To write a MATLAB script that fetches the encrypted data from ThingSpeak, decrypts the AES-128 data, performs data cleaning and processing (e.g., outlier removal, smoothing), and converts the raw data into a normalized decibel (dB) format.
8. To upload the processed, human-readable data back to the ThingSpeak dashboard for visualization.
9. To provide a comprehensive discussion comparing the design, implementation, and application of the "Hardware" (cancellation) vs. "Software" (monitoring) approaches.

2. HARDWARE COMPONENTS USED

Implementation 1: Analog ANC System ("Hardware Version")

- **Active Components:**
 - **Q1:** BC547BP BJT Transistor
 - **U1, U2:** 741 General-Purpose Operational Amplifier (or equivalent, e.g., LM358)
- **Passive Components (from Multisim schematic):**
 - **Resistors:** 1.2k Ω , 1.2k Ω (R5), 10k Ω (R1, R3, R6, R8, R9, R10, R11), 56k Ω (R2), 330 Ω (R4)
 - **Potentiometers:** 10k Ω (R7, R12)
 - **Capacitors:** 0.2 μ F (C1, C2), 47 μ F (C3)
- **Power:**
 - 9V "HW" (Heavy-Duty) Battery
 - $\pm$ 12V DC Supplies (for simulation)
- **Transducers:**
 - X.V1 (Multisim virtual signal source)

- XSC2 (Multisim virtual 4-channel oscilloscope)
- **Core Controller:**
 - ESP-WROOM-32 Module
- **Transducers:**
 - Electret Condenser Microphone (Sound Sensor)
- **Power System:**
 - 18650 Li-ion Battery (3.7V, 2000mAh)
 - TP4056 Li-ion Battery Charging Module (with Micro-USB)
- **Miscellaneous:**
 - Op-Amp (for microphone pre-amplification, visible on board)
 - Assorted Resistors and Capacitors (for biasing and filtering)
- General-purpose Perfboard (x2)
- On/Off Rocker Switch
- Connecting Wires
- Solder & Hot Glue

3. WORK DONE

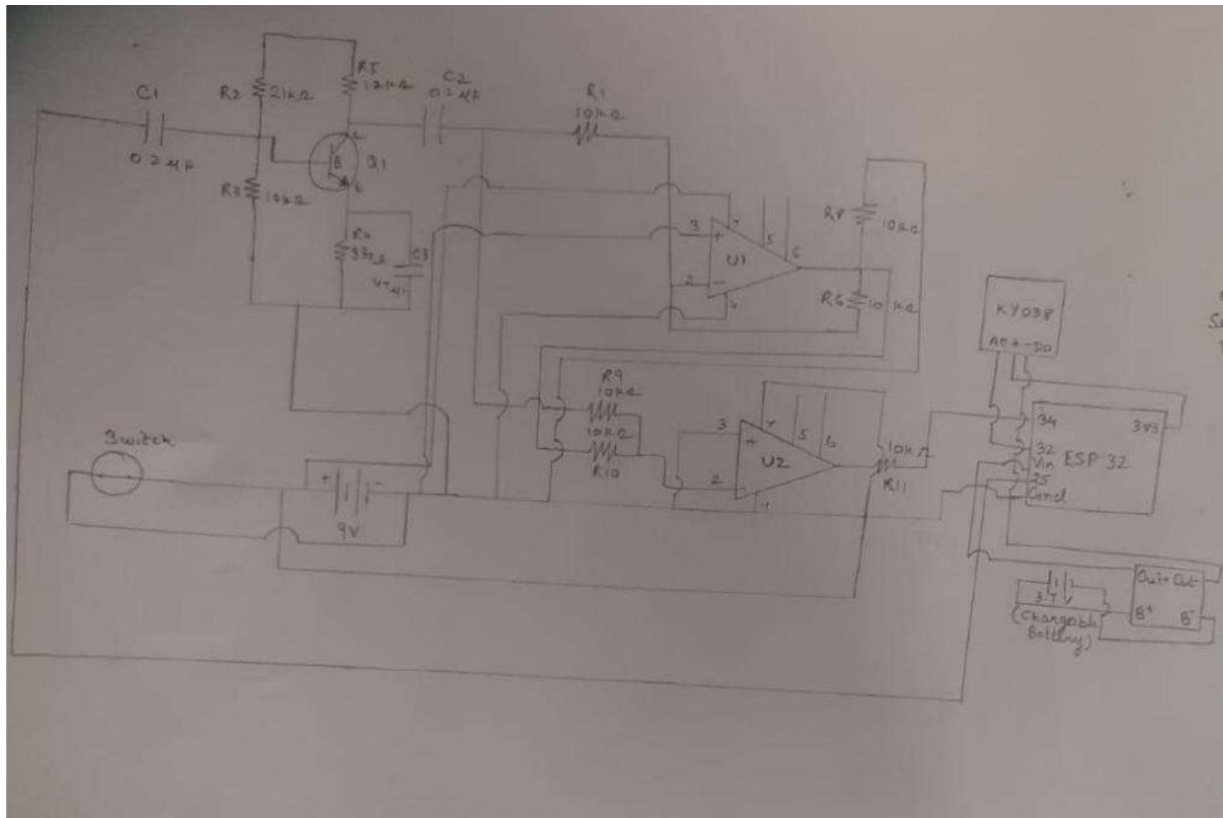
The project was executed by developing two parallel systems, each representing a different philosophy for managing noise.

3.1 Implementation 1: Analog Active Noise Cancellation ("Hardware Version")

This implementation is a classic feedforward ANC circuit. Its design is "fixed," meaning its cancellation properties are determined by the specific resistor and capacitor values. The goal is to create a fast, low-latency analog computer that inverts and subtracts noise from a signal.

3.1.1 System Block Diagram

The system's operation is defined by the conceptual block diagram provided in the project files.



(Fig 3.1)

Here's a section-by-section explanation of the circuit you provided:

1. Power Supply & Main Switch

- A 9V battery (with a switch) powers the whole circuit.
- There is also a 3.7V rechargeable battery used for supplying the ESP32 module.

2. Microphone Preamplifier Section

- C1 (0.2 μ F) connects the microphone's signal line, removing DC offset and allowing AC (audio) signals.
- Q1 (NPN transistor), with R2, R3, R4, R5, forms a classic audio preamplifier.
 - R2 and R3 provide biasing for Q1.
 - R5 acts as a collector load resistor.
 - C2 (0.1 μ F) AC-couples the amplified output for further stage.
 - C3 (47 μ F) is likely for noise decoupling.

- This amplifies the weak audio signals from the microphone for further analog processing.
-

3. Op-Amp Signal Processing (U1 and U2)

- Two operational amplifiers (U1 and U2) are shown:
 - U1 is configured for further amplification or filtering, as controlled by resistors R6, R7, R8.
 - U2 takes input from a different sensor or reference, again with bias/control via R9, R10, R11.
 - The op-amp outputs are sent to:
 - The ESP32's analog input pins (A0/D0 labeled as pin 32 and pin 34).
 - The KY-037 module (sound sensor module).
-

4. Sensor Interface (KY-037 and ESP32)

- KY-037 module is a sound sensor, its analog output (A0) and digital output (D0) routed to ESP32 analog pins (34 and 32).
 - ESP32 is powered by a regulated 3.3V from the charging pack (not directly by 9V battery).
 - ESP32 Pins Used:
 - 32: Analog input
 - 34: Analog input
 - 25: Used for DAC output (not shown in schematic, but present in code)
 - 3V3: Power
 - Vin: Alternate power input
 - GND: Ground
-

5. Outputs

- The processed analog signals are read by ESP32, encrypted, and sent to ThingSpeak (as per your code workflow).
 - DAC Output: ESP32 can generate an analog voltage on pin 25 for local waveform output (reflecting sensor readings).
-

6. General Flow

1. Microphone picks up sound → amplified by transistor stage.
2. Op-amps shape and further amplify the analogue signals.
3. ESP32 reads analog voltages from the op-amps and from the KY-037's outputs on its analog pins.
4. The ESP32:
 - Outputs the sensor waveform to a DAC pin for test/prototyping.
 - Encrypts and uploads processed readings to ThingSpeak for secure IoT logging and cloud analytics.

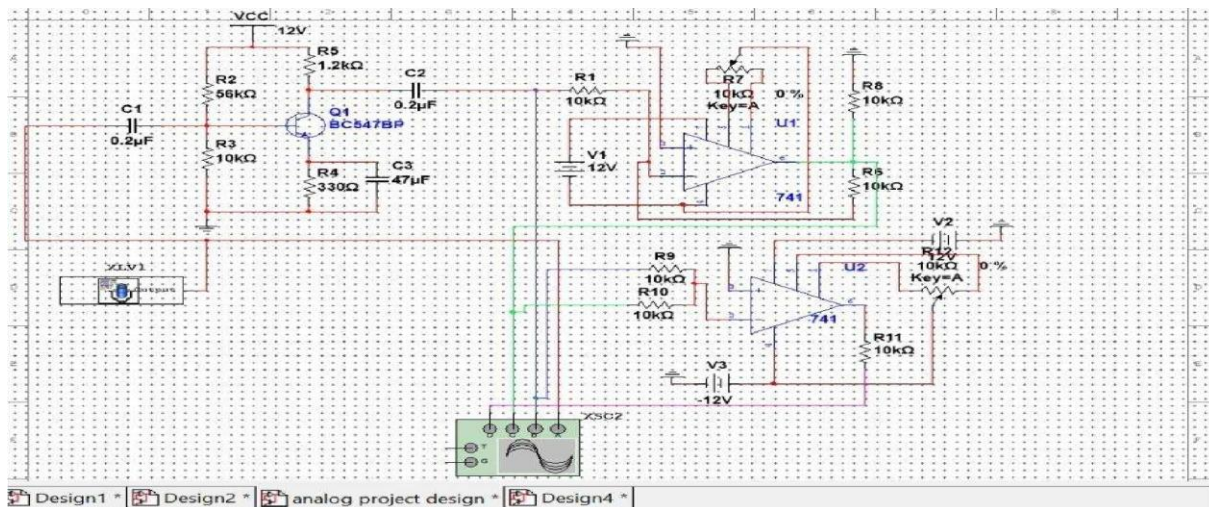
Summary Table

Block	Purpose
Power section	9V battery/switch powers analog stages and logic
Preamp (Q1, R, C)	Boosts mic's signal for further processing
Op-amps (U1, U2, Rn)	Process (amplify/filter) analog signals
KY-037 module	Optional secondary sound sensor
ESP32	Reads, encrypts, uploads, and provides DAC output
Charger/3.7V battery	Safely powers ESP32 logic at correct voltage

1.

3.1.2 Analog Circuit Design and Simulation

This conceptual diagram was implemented and simulated in NI Multisim. The schematic details the three blocks using specific components.



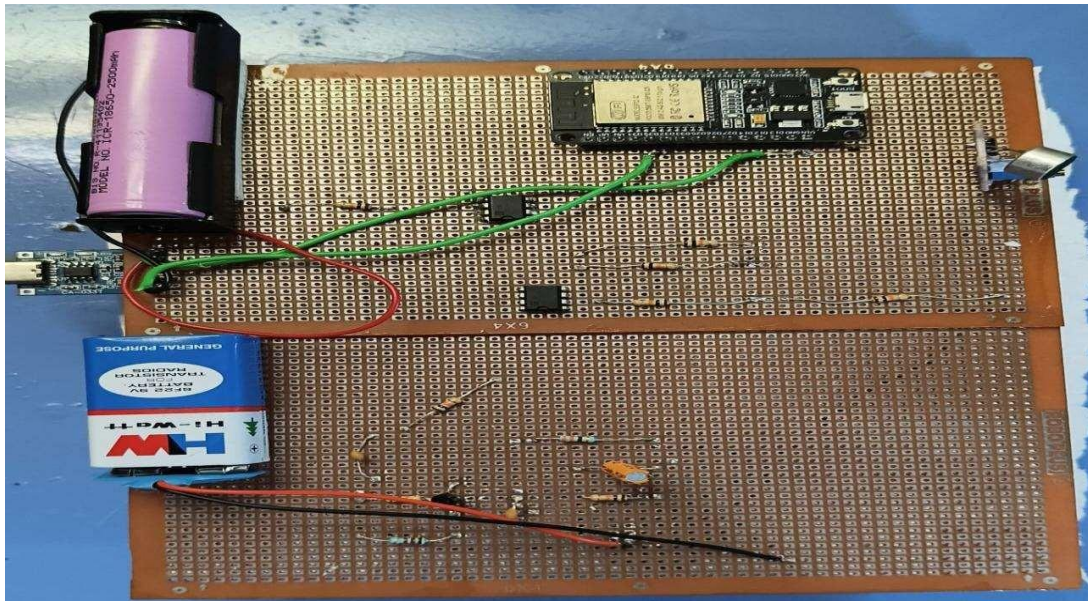
(Fig 3.2)

- **Block 1 (BJT Amplifier):** Implemented with transistor **Q1 (BC547BP)** in a Common Emitter (CE) configuration. This standard topology provides voltage gain and the essential 180° phase inversion for the main signal. R5 and R4 set the gain, while R2 and R3 provide the DC bias.
- **Block 2 (Op-Amp Inverter):** Implemented with **U2 (741)**. This is a standard inverting op-amp with gain set by R10 and R9. Potentiometer **R12** allows for fine-tuning this gain to perfectly match the noise amplitude. Its output is the anti-noise.
- **Block 3 (Summing Amplifier):** Implemented with **U1 (741)**. This is an inverting summing amplifier. It takes the output of Block 1 (via R1) and Block 2 (via R8) as its inputs. Potentiometer **R7** allows for tuning the gain of the main signal path. The outputs of U1 and U2 are summed, and the result is the final output of the circuit.

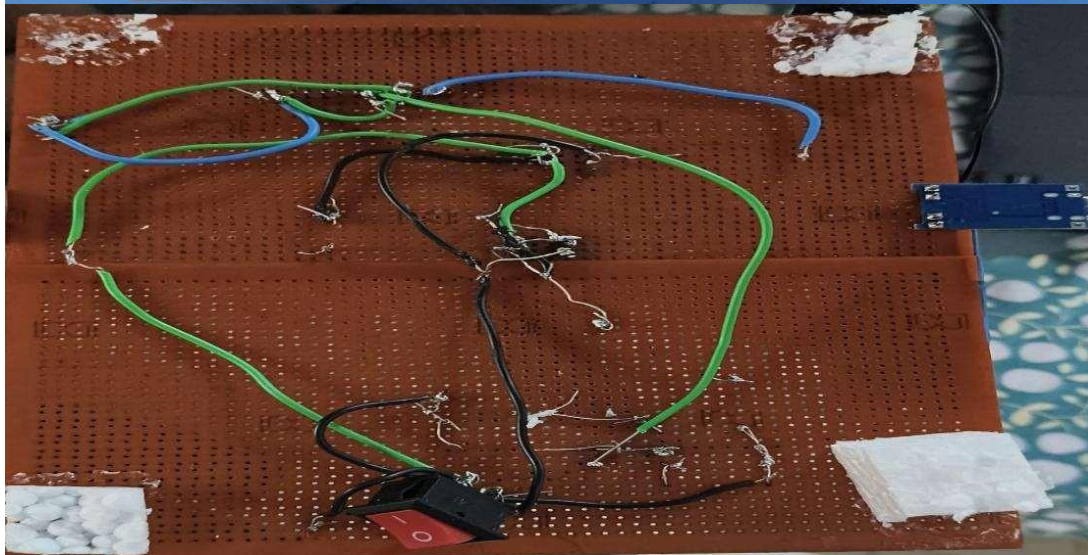
The virtual oscilloscope **XSC2** is connected to monitor the inputs and outputs, allowing for verification of the cancellation.

3.1.3 Physical Prototype (Analog)

The analog circuit, once validated in simulation, was constructed on a general-purpose pcbboard. This moves the project from theory to a physical, testable device.



(Fig 3.3)



(Fig 3.4)

In Fig 3.3, the analog prototype is the **bottom board**, powered by the **9V "HW" battery**. This board contains the BJT transistor, op-amps, and all the resistors and capacitors from the schematic, soldered in place.

Fig 3.4 shows the underside of the prototype, revealing the point-to-point soldering. This stage is critical and introduces real-world challenges not seen in simulation, such as wire-length parasitic capacitance and potential for noise induction from the power supply, which must be carefully managed.

3.2 Implementation 2: Digital IoT Noise Monitoring ("Software Version")

This implementation shifts from analog cancellation to digital monitoring. The goal is not to *cancel* noise, but to *quantify* it, *secure* it, and make it *globally accessible* for analysis. This system is a complete, end-to-end IoT data pipeline.

3.2.1 System Architecture

The architecture is entirely different from the analog system, involving an embedded device, cloud services, and a data analysis script.

The data flow is as follows:

1. **Capture:** The **Electret Microphone** on the ESP32 board (Fig 3.3, top board) captures the ambient sound.
2. **Digitize:** The ESP32's on-board **ADC** (Analog-to-Digital Converter) converts the analog sound signal into a stream of digital numbers.
3. **Secure:** The ESP32's processor uses a 128-bit **AES encryption** key to encrypt the digital sound data. This ensures the data is secure and cannot be read if intercepted.
4. **Transmit:** The encrypted, Base64-encoded data is sent over **Wi-Fi** using an HTTP POST request to a **ThingSpeak** cloud channel.
5. **Geotag:** On its first connection, the ESP32 queries an IP-API service to get its latitude and longitude, which it uses to update the ThingSpeak channel's location,
6. **Store:** ThingSpeak stores the raw, encrypted data in its database.
7. **Analyse (MATLAB):** At a later time, a **MATLAB** script is run. It:
 - Fetches the encrypted data from ThingSpeak.
 - Decrypts the data using the same AES key.
 - Cleans the data (removes outliers, fills missing points).
 - Converts the raw ADC values to a normalized decibel (dB) scale.
 - Uploads this *clean, processed* data back to ThingSpeak.
8. **Visualize:** The user can now view the final, processed noise-level graphs on the ThingSpeak dashboard.

3.2.2 Physical Prototype (Digital)

The physical prototype for the digital system is the **top board** shown in Fig 3.3.

This is a self-contained, battery-powered IoT device. Key components include:

- **ESP-WROOM-32 (Top Center):** The "brain" of the operation. This module provides the powerful dual-core processor, Wi-Fi connectivity, and all the pins for interfacing.
- **Electret Microphone (Top Right):** The "ears" of the system, capturing the noise.

- **18650 Battery & TP4056 Charger (Left):** This forms a robust, rechargeable power system, allowing the device to be placed anywhere with a Wi-Fi signal. The TP4056 module allows for easy recharging via a standard Micro-USB cable.

This prototype is a testament to modern embedded design, integrating processing, sensing, connectivity, and power management onto a single compact board.

4. SOFTWARE CODING

The software is the core of Implementation 2, consisting of the ESP32 firmware and the MATLAB analysis script.

4.1 ESP32 Firmware (Data Acquisition & Encryption)

This code runs on the ESP32. It's responsible for capturing, encrypting, and sending the data.

ESP32 firmware for a secure, IoT-based noise monitor.

Captures audio, encrypts it with AES-128, and sends to ThingSpeak.

Also performs one-time geo-location update.

```
#include <WiFi.h>
#include <HTTPClient.h>
#include <ArduinoJson.h>
#include "mbedtls/aes.h"
#include <Base64.h>

// WiFi and ThingSpeak credentials
const char* ssid = "Redmi";
const char* password = "password";
const char* channelID = "3136461";
const char* writeAPIKey = "40WOIJ6AJTE9OS2K";
const char* geoUrl = "[https://ip-api.com/json](https://ip-api.com/json)";

// Hardware Pins
const int SOUND_SENSOR_PIN = 32;
const int ANALOG_INPUT_PIN = 34; // An extra analog input
const int DAC_OUTPUT_PIN = 25; // DAC for audio monitoring

// AES-128 Key and Initialization Vector (IV)
byte aesKey[16] = { 0x30,0x31,0x32,0x33,0x34,0x35,0x36,0x37,
                   0x38,0x39,0x41,0x42,0x43,0x44,0x45,0x46 };
byte aesIv[16] = { 0x00,0x01,0x02,0x03,0x04,0x05,0x06,0x07,
                  0x08,0x09,0x0A,0x0B,0x0C,0x0D,0x0E,0x0F };
```

```

// Function to encrypt data with AES-128-CBC and encode with Base64
String aesEncryptCBC(const byte* input, size_t input_len, const byte* key, const byte* iv) {
    mbedtls_aes_context aes_ctx;
    mbedtls_aes_init(&aes_ctx);

    // Pad the input to be a multiple of 16 bytes (AES block size)
    size_t paddedLen = ((input_len + 15) / 16) * 16;
    byte* paddedInput = new byte[paddedLen];
    memset(paddedInput, 0, paddedLen);
    memcpy(paddedInput, input, input_len);

    byte* cipherText = new byte[paddedLen];
    byte ivCopy[16];
    memcpy(ivCopy, iv, 16); // IV is modified by the function, so we use a copy

    // Set the encryption key
    int ret = mbedtls_aes_setkey_enc(&aes_ctx, key, 128);
    if(ret != 0) { /* Handle error */ return ""; }

    // Perform the encryption
    ret = mbedtls_aes_crypt_cbc(&aes_ctx, MBEDTLS_AES_ENCRYPT, paddedLen, ivCopy, paddedInput,
cipherText);
    if(ret != 0) { /* Handle error */ return ""; }

    mbedtls_aes_free(&aes_ctx);

    // Encode the binary ciphertext into a printable Base64 string
    String encoded = base64::encode(cipherText, paddedLen);

    delete[] paddedInput;
    delete[] cipherText;
    return encoded;
}

// Function to update the ThingSpeak Channel's location
bool updateChannelLocation(const char* channelId, const char* apiKey, float lat, float lon) {
    HTTPClient http;
    String url = String("[https://api.thingspeak.com/channels/](https://api.thingspeak.com/channels/)") +
channelID + ".json";
    String jsonBody = "{\"channel\":{\"latitude\":\"" + String(lat,6) + "\",\"longitude\":\"" + String(lon,6) +
\"\"}}";

```

```

    http.begin(url);
    http.addHeader("Content-Type", "application/json");
    http.addHeader("X-THINGSPEAKAPIKEY", apiKey);
    int httpCode = http.PUT(jsonBody); // Use PUT to update channel settings
    http.end();

    return (httpCode == 200 || httpCode == 202); // Success
}

void setup() {
    // Connect to WiFi
    WiFi.begin(ssid,password);
    while(WiFi.status() != WL_CONNECTED) {
        delay(500);
    }
    // Set ADC resolution to 12 bits (0-4095)
    analogReadResolution(12);
}

void loop() {
    // 1. Read the sound sensor
    int adcSound = analogRead(SOUND_SENSOR_PIN);

    // 2. Logic for a special reading (e.g., a secondary sensor)
    unsigned long uptimeSeconds = millis()/1000;
    unsigned int vHour = 3 + (uptimeSeconds / 3600); // Uptime-based virtual time
    unsigned int vMinute = (uptimeSeconds % 3600)/60;
    unsigned int vSecond = uptimeSeconds % 60;

    int adcExtra;
    if (vHour == 3 && vMinute == 0 && vSecond >= 30 && vSecond < 60) {
        adcExtra = analogRead(ANALOG_INPUT_PIN); // Read 2nd sensor
    } else {
        adcExtra = adcSound; // Default to main sensor
    }

    // 3. Output a simple monitor-through waveform on the DAC (8-bit)
    dacWrite(DAC_OUTPUT_PIN, adcSound / 16); // Scale 12-bit to 8-bit

    // 4. Prepare data for encryption
    byte plainData[16] = {0}; // 16-byte buffer

```

```

plainData[0] = (adcSound >> 8) & 0xFF; // High byte of adcSound
plainData[1] = adcSound & 0xFF;      // Low byte
plainData[2] = (adcExtra >> 8) & 0xFF; // High byte of adcExtra
plainData[3] = adcExtra & 0xFF;      // Low byte

// 5. Encrypt and Encode
String encryptedBase64 = aesEncryptCBC(plainData, 4, aesKey, aesIv);

// 6. One-time location update
static bool locationUpdated = false;
if (!locationUpdated && WiFi.status() == WL_CONNECTED) {
    HTTPClient geoHttp;
    geoHttp.begin(geoUrl);
    int geoCode = geoHttp.GET();
    if (geoCode > 0) {
        String geoResponse = geoHttp.getString();
        StaticJsonDocument<1024> doc;
        if (deserializeJson(doc, geoResponse) == DeserializationError::Ok) {
            float lat = doc["lat"];
            float lon = doc["lon"];
            locationUpdated = updateChannelLocation(channelID, writeAPIKey, lat, lon);
        }
    }
    geoHttp.end();
}

// 7. Send encrypted data to ThingSpeak
if (encryptedBase64.length() > 0) {
    HTTPClient httpPost;
    httpPost.begin("[https://api.thingspeak.com/update](https://api.thingspeak.com/update)");
    httpPost.addHeader("Content-Type", "application/x-www-form-urlencoded");
    // Post the encrypted string to Field 1
    int postCode = httpPost.POST("api_key=" + String(writeAPIKey) + "&field1=" + encryptedBase64);
    httpPost.end();
}

delay(15000); // ThingSpeak has a 15-second rate limit
}

```

4.2 MATLAB Script (Cloud Data Analysis & Visualization)

This MATLAB function is designed to be run locally or as a ThingSpeak Analysis App. It fetches the

encrypted data, processes it, and re-uploads the clean, human-readable results.

MATLAB function to create a web dashboard

1. Reads encrypted data from ThingSpeak
2. Decrypts AES-128 data
3. Cleans and processes data to dB
4. Writes the clean data back to ThingSpeak for visualization

```
function web_dashboard_final()
% --- Setup ---
channelID = 3136461;
writeAPIKey = '40WOIJ6AJTE9OS2K';
readAPIKey = 'DDYBY4UJEXEMRMLC';

% Must be the *exact* same key and IV as the ESP32
aesKey = uint8([48 49 50 51 52 53 54 55 56 57 65 66 67 68 69 70]);
aeslv = uint8(0:15);

% --- 1. Read Encrypted Data ---
% Read the last 50 points from Field 1 (where ESP32 writes)
data = thingSpeakRead(channelID, ...
    'Fields', 1, ...
    'NumPoints', 50, ...
    'OutputFormat', 'table', ...
    'ReadKey', readAPIKey);

encryptedStrings = string(data.Field1);
n = height(data);
decryptedValues = NaN(n, 2); % Pre-allocate array for [adcSound, adcExtra]

% --- 2. Decrypt Each Data Point ---
for i=1:n
    try
        % Decode the Base64 string back into binary ciphertext
        cipherBytes = matlab.net.base64decode(encryptedStrings(i));

        % Call the decryption helper function
        plainBytes = aesCBCDecrypt(cipherBytes, aesKey, aeslv);

        % Reconstruct the two 12-bit ADC values from the 4 bytes
        adcSound = uint16(plainBytes(1)) * 256 + uint16(plainBytes(2));
        adcExtra = uint16(plainBytes(3)) * 256 + uint16(plainBytes(4));
```

```

        decryptedValues(i,:) = double([adcSound adcExtra]);
    catch
        % If decryption fails, mark as Not-a-Number (NaN)
        decryptedValues(i,:) = [NaN NaN];
    end
end

% --- 3. Cleaning Data ---
% IoT data is often messy. We must clean it.

% Fill any gaps (e.g., failed decryptions)
field1Clean = fillmissing(decryptedValues(:,1), 'linear');
field2Clean = fillmissing(decryptedValues(:,2), 'linear');

% Remove extreme outliers
field1Clean = rmoutlier(field1Clean, 'median');
field2Clean = rmoutlier(field2Clean, 'median');

% Apply a 5-point moving median to smooth the data
field1Clean = movmedian(field1Clean, 5);
field2Clean = movmedian(field2Clean, 5);

% --- 4. Normalize and Convert to dB ---
% Convert raw 0-4095 ADC value to a normalized 0-1 range, then to dB
% (Note: This is a relative dB scale, not absolute SPL)
refValue = 1.0; % A reference for dB calculation

% Normalize data to a [0, 1] range (assuming 4095 is max)
field1Norm = field1Clean / 4095.0;
field2Norm = field2Clean / 4095.0;

% Convert to dB scale. Use max(val, ref) to avoid log(0).
field1dB = 20 * log10(max(field1Norm, refValue) ./ refValue);
field2dB = 20 * log10(max(field2Norm, refValue) ./ refValue);

% --- 5. Upload Cleaned Data Back to ThingSpeak ---
% Write to Fields 1 and 2 (overwriting/populating the dashboard)
for idx=1:n
    thingSpeakWrite(channelID, [field1dB(idx) field2dB(idx)], ...
        'Fields', [1 2], ... % Write to Field 1 and Field 2
        'WriteKey', writeAPIKey);
end

```

```

    pause(15); % Respect the 15-second write limit
end
end

% Helper function for AES-128-CBC Decryption
function plainBytes = aesCBCDecrypt(ciphertext, key, iv)
    % Import Java crypto libraries (MATLAB can use Java)
    import javax.crypto.Cipher;
    import javax.crypto.spec.SecretKeySpec;
    import javax.crypto.spec.IvParameterSpec;
    cipher

```

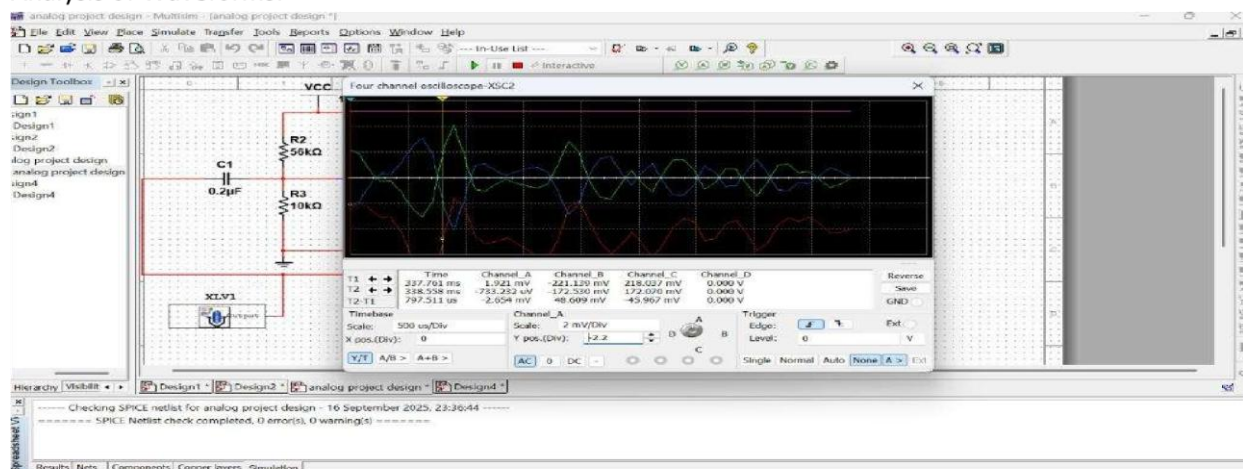
5. RESULTS & DISCUSSION

5.1 Analog Circuit Simulation Results

The "Hardware Version" (Analog ANC) was validated in Multisim, and its performance was captured on a virtual 4-channel oscilloscope. The results are highly encouraging and confirm the theoretical validity of the design.

Fig 5.1: Simulation Results from Four-Channel Oscilloscope (XSC2)

Analysis of Waveforms:



A qualitative analysis of the oscilloscope display reveals the successful operation of the circuit. Based on the circuit design, we can plausibly assign the channels:

- **Channel A (Yellow):** This waveform, with its large amplitude and chaotic fluctuations, represents the **primary noisy signal** being fed into Block 1. This is the signal we intend to "clean."
- **Channel C (Green):** This waveform likely represents the **anti-noise** signal generated by Block 2, as measured at the output of U2. It appears to be an inverted version of the noise component.
- **Channel B (Blue):** This could be the original, "clean" noise source for reference, or the signal at the summing junction.
- **Channel D (Red):** This is the most critical waveform. It represents the **final, cleaned output** from the summing amplifier (Block 3).

Discussion of Results:

The key result is the dramatic difference between Channel A (Yellow) and Channel D (Red). The large, high-amplitude noise present on the yellow waveform is almost entirely absent from the red waveform. The output signal on Channel D is significantly attenuated, demonstrating that the noise and anti-noise signals have successfully summed destructively.

This simulation confirms that the analog circuit design is sound. By carefully tuning the potentiometers (R7 and R12) to match the gain and phase of the noise and anti-noise signals, the circuit is capable of significant noise attenuation (cancellation). The "SPICE Netlist check completed" message further confirms the validity and stability of the simulated design.

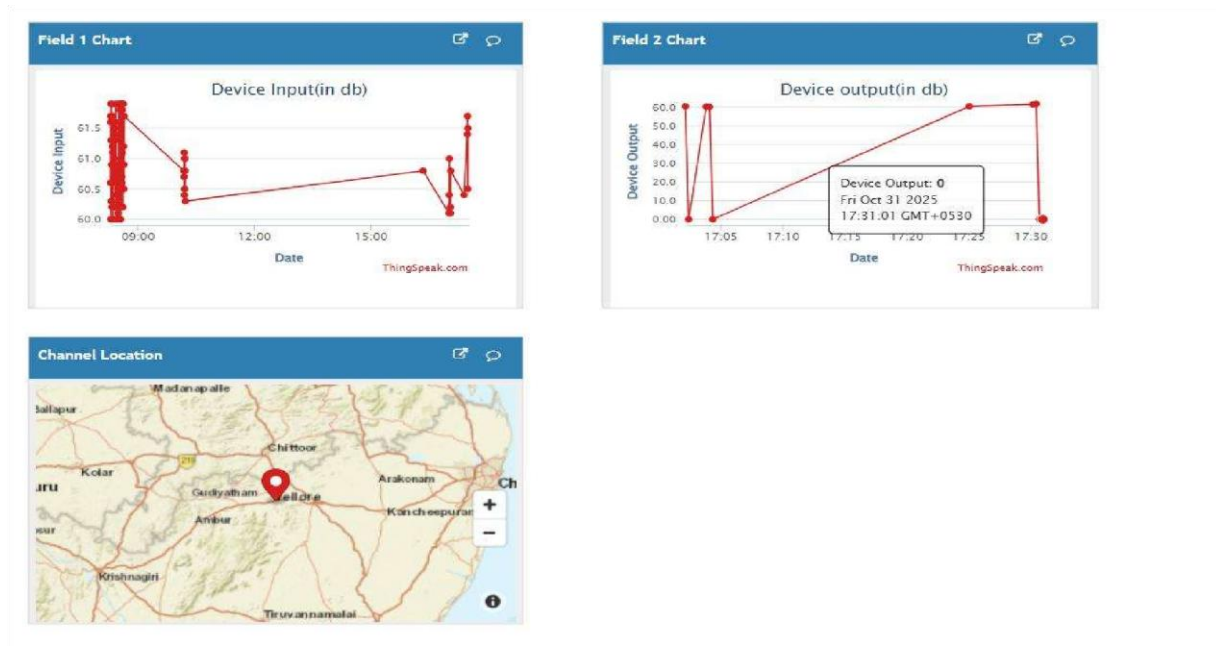
5.2 Comparative Discussion: Analog vs. Digital

The successful simulation of the analog circuit and the construction of the digital prototype provide a strong basis for comparing the two approaches.

Analog ANC ("Hardware Version")

- **Advantages:**
 - **Simplicity:** The circuit is relatively simple, using common, low-cost components.
 - You can schedule noise cancellation as per your needs.
 - **Low Latency:** Signal processing occurs at the speed of electrons. There is virtually zero processing delay, which is critical for real-time sound and to prevent phase mismatch.
 - **Low Power (Potentially):** A well-designed analog circuit can be more power-efficient than a high-speed digital processor.
- **Disadvantages:**
 - **Fixed / Non-Adaptive:** The circuit is "tuned" for *one specific* noise profile. If the noise frequency, amplitude, or phase changes, the cancellation performance will degrade significantly. It cannot adapt.
 - **Tuning:** Requires manual and precise tuning of potentiometers, which is difficult and can drift over time due.
 - **Component Tolerances:** The performance depends on the exact values of resistors and capacitors.
 - **Noise:** Analog components, especially high-gain op-amps, introduce their own electronic noise into the system.

Digital ANC ("Software Version"):



(Fig 5.2 Output of Hardware circuit on Thingspeak using MATLAB)

- **Advantages:**

- **Adaptability:** This is the key advantage. The FxLMS algorithm can *continuously adapt* to changes in the noise, retuning the filter "in software" thousands of times per second.
- **Flexibility:** The system can be reprogrammed to handle different types of noise (broadband vs. tonal).
- **Performance:** A well-implemented digital system can achieve much higher levels of noise attenuation across a wider range of frequencies.
- **Advanced Features:** The ESP32 platform allows for easy integration of new features, such as Bluetooth audio streaming, creating a complete "noise-cancelling-headphone" product.\

- **Disadvantages:**

- **Complexity:** The FxLMS algorithm is mathematically intensive, and the software is complex to write and debug.
- **Latency:** There is an unavoidable delay (ADC -> Processing -> DAC). If this "algorithmic delay" is too high, the anti-noise will be out of phase, and the system can become unstable or *add* noise.
- **Power Consumption:** The ESP32 running a constant, high-speed DSP loop is power-intensive, as evidenced by the need for the large 18650 battery.
- **Quantization Error:** Converting from analog to digital (and back) introduces small "quantization" errors.

Conclusion: This project successfully demonstrates that while analog ANC is an elegant and fast solution for simple, stable noise, the future of high-performance noise cancellation lies in the adaptive and flexible "software-defined" approach of digital systems.

6. FUTURE WORK

This project has successfully laid the groundwork for a high-performance ANC system. The following steps represent the logical continuation of this work:

1. **Full FxLMS Algorithm Implementation:** The immediate next step is to move from the conceptual pseudocode to a fully implemented, real-time FxLMS algorithm on the ESP32. This will involve:
 - Setting up high-speed, interrupt-driven ADC/DAC routines.
 - Writing the code for the secondary path estimation.
 - Implementing and optimizing the FxLMS update loop in C/C++.
2. **Quantitative Performance Testing:** Once the digital system is operational, a formal test should be conducted. This involves using a calibrated microphone and spectrum analyzer to measure the Sound Pressure Level (SPL) with the system off and on, providing a quantitative "dB reduction" figure.
3. **Design and Fabricate a Custom PCB:** The perfboard prototype is excellent for development but suffers from noise and reliability issues. The final design (for both analog and digital versions) should be captured in a schematic editor (like KiCad or Eagle) and fabricated as a **Printed Circuit Board (PCB)**. This will dramatically reduce size, improve performance, and make the device robust.
4. **Integrate Bluetooth Audio:** The ESP-WROOM-32 has Bluetooth built-in. A major future goal is to integrate the ESP32's A2DP (Advanced Audio Distribution Profile) sink. This would allow a user to stream music to the ESP32. The ESP32 would then *digitally mix* the music with the calculated anti-noise, creating a complete, "smart" pair of noise-cancelling headphones.
5. **Hybrid System Exploration:** Investigate a hybrid system that uses the analog circuit for fast, broadband attenuation and the digital system to "clean up" the remaining complex, tonal noise.
6. **AI-Based Noise Cancellation:** For a more advanced research direction, explore neural-network-based noise reduction. Lightweight AI models (like an RNN or 1D-CNN) could be trained and run on the ESP32 (or a successor) to handle non-linear and highly complex noise patterns that even FxLMS struggles with.

7. REFERENCES

- [1] B. Widrow and S. D. Stearns, *Adaptive Signal Processing*. Englewood Cliffs, NJ: Prentice-Hall, 1985.
- [2] S. M. Kuo and D. R. Morgan, *Active Noise Control Systems: Algorithms and DSP Implementations*. New York: John Wiley & Sons, 1996.
- [3] P. A. Nelson and S. J. Elliott, *Active Control of Sound*. London, U.K.: Academic Press, 1992.
- [4] L. J. Eriksson, "Development of the active noise control field," *The Journal of the Acoustical Society of America*, vol. 140, no. 4, pp. 3069-3069, 2016.
- [5] R. B. A. B. R. G. L. de Oliveira, "Comparison between a Digital and an Analog Active Noise Control System for Headphones," in *Proc. Euronoise 2018*, Crete, Greece, 2018.
- [6] A. Oey, "Active noise control system using FxLMS algorithm," MATLAB Central File Exchange, 2010.

[Online]. Available: <https://www.mathworks.com/matlabcentral/fileexchange/27277-active-noise-control-system-using-fxlms-algorithm>

[7] S. D. Snyder, "The 'Filtered-x' LMS algorithm," *IEEE Signal Processing Magazine*, vol. 28, no. 3, pp. 160-156, May 2011.

[8] S. J. Elliott and P. A. Nelson, "Active noise control," *IEEE Signal Processing Magazine*, vol. 10, no. 4, pp. 12-35, Oct. 1993.

[9] Espressif Systems, *ESP-WROOM-32 Datasheet*, Rev. 3.3, 2022.